

PORTADA

Análisis y Diseño de Sistemas II

**Semana 14 — Antipatrones de Diseño de Software y Laboratorio de Cierre de la
Unidad 5**

Universidad de Antioquia · Ingeniería de Sistemas



APERTURA

**La misma solución que evitó un problema hace un año
hoy es la que nadie se atreve a tocar**

TRANSICIÓN

De los patrones que funcionan a lo que sale mal sin ellos

El martes vieron **Strategy, Observer y Command**; el miércoles, **State, Template Method e Iterator**. Con eso se completó el catálogo de patrones de comportamiento GoF y las cuatro familias de patrones de esta unidad: GRASP (semana 10), creacionales (semana 12), estructurales (semana 13) y comportamiento (semana 14).

Hoy no se ve un patrón nuevo. Se ve qué pasa cuando esos patrones **no se aplican, se aplican mal, o se ignoran**: los antipatrones. La segunda mitad de la sesión es el **laboratorio de cierre de la Unidad 5 completa**, donde cada equipo busca antipatrones en su propio proyecto y repasa los patrones que ya aplicó.

CONCEPTO

Qué es un antipatrón

Un **patrón de diseño** es una solución probada y recomendada a un problema recurrente — los GRASP y los GoF vistos en las últimas cinco semanas son ejemplos. Un **antipatrón** es distinto: es una solución que a primera vista parece razonable, e incluso resuelve el problema inmediato, pero que se repite tanto en la práctica de la industria que terminó documentada con nombre propio — y que, sostenida en el tiempo, produce **más problemas de los que resolvió**.

Casi ningún equipo elige un antipatrón a propósito. Aparece de forma gradual: una decisión razonable en el momento, que nunca se revisa después, y que termina consolidándose como "la forma normal de hacer las cosas" en ese proyecto.

CONCEPTO

Por qué se ven al final de la unidad, y no al principio

Reconocer un antipatrón exige primero saber cómo se ve una solución bien aplicada. Por eso los antipatrones se ven después de las nueve GRASP, los cinco creacionales, los seis estructurales y los seis de comportamiento GoF — no antes.

Varios de los que se ven hoy ya aparecieron de pasada, sin nombre propio, en semanas anteriores del curso: hoy se formalizan como antipatrones con nombre, y se conecta cada uno con el principio o patrón que, de haberse aplicado bien, lo habría evitado.

CONCEPTO

God Object / God Class

Un **God Object** (o God Class) es una clase que acumula demasiadas responsabilidades y termina conociendo o controlando una porción desproporcionada del sistema. En vez de colaborar con otras clases especializadas, todo pasa por ella: validaciones, cálculos, persistencia, notificaciones, reportes.

Ya se insinuó dos veces antes en el curso, sin llamarlo por su nombre: en la **semana 7**, al hablar de una clase con demasiadas razones para cambiar (baja cohesión); y en la **semana 10**, como "**God Controller**", cuando un único Controller GRASP concentraba toda la lógica de una operación en vez de delegarla a colaboradores.



CONTRA EJEMPLO

Una sola clase haciendo de todo

```
class GestorMatricula:
    def validar_prerrequisitos(self, estudiante, curso): ...
    def calcular_costo_semestre(self, estudiante): ...
    def guardar_en_base_de_datos(self, matricula): ...
    def generar_pdf_recibo(self, matricula): ...
    def enviar_correo_confirmacion(self, estudiante): ...
    def generar_reporte_financiero(self, periodo): ...
    def validar_cupos_disponibles(self, curso): ...
# una clase, siete responsabilidades sin relación entre sí
```

`GestorMatricula` mezcla reglas de negocio, acceso a datos, generación de documentos, notificaciones y reportería. Cualquier cambio en cualquiera de esas áreas obliga a tocar la misma clase, y cualquier prueba unitaria arrastra todas las demás responsabilidades.

CONCEPTO · RELACIÓN CON LO VISTO

God Object es la negación directa de High Cohesion y SRP

High Cohesion (GRASP, semana 10) pide asignar responsabilidades de modo que una clase se mantenga enfocada y manejable. **Single Responsibility** (SOLID, semana 8) pide que una clase tenga una sola razón para cambiar. God Object es lo que ocurre cuando ninguna de las dos se respeta de forma sostenida: cada responsabilidad nueva se agrega a la misma clase porque "ya está ahí y es fácil", en vez de crear una colaboradora.

Corrección típica: identificar las responsabilidades mezcladas y extraerlas en clases nuevas, usando **Information Expert** (¿quién tiene los datos para hacer cada cosa?) para decidir a cuál colaboradora le corresponde cada una.

CONCEPTO

Spaghetti Code

Spaghetti Code es código con flujo de control enredado, sin una estructura modular clara — como los hilos de un plato de espagueti, es difícil seguir una sola "hebra" de lógica sin perderse en las demás, y modificar una parte suele romper otra sin que sea obvio por qué.

Aparece con frecuencia como cadenas largas de condicionales anidados (`if/elif` profundos o `switch` gigantes) que deciden el comportamiento según un tipo o un estado, en vez de delegar esa decisión a una clase especializada.



CONTRA EJEMPLO

Condicionales anidados donde ya había un patrón para esto

```
def calcular_envio(tipo, estado_pedido, peso):  
    if tipo == "aereo":  
        if estado_pedido == "pendiente":  
            ...  
        elif estado_pedido == "en_transito":  
            ...  
    elif tipo == "terrestre":  
        if estado_pedido == "pendiente":  
            ...  
        elif estado_pedido == "en_transito":  
            ...  
    elif tipo == "maritimo":  
        ... # y sigue creciendo con cada tipo nuevo
```

Esta es la estructura que **Strategy** (martes) resuelve delegando el algoritmo variable (el tipo de envío) a una clase intercambiable, o que **State** (miércoles) resuelve delegando el comportamiento dependiente del estado a una jerarquía de clases estado. Spaghetti Code, muchas veces, es la señal de que un patrón de comportamiento ya visto en la unidad hacía falta y no se aplicó.

CONCEPTO

Golden Hammer

Golden Hammer es aplicar la misma solución, patrón o tecnología favorita a todos los problemas, sin evaluar si realmente encaja — la idea popular de "si solo tienes un martillo, todo parece un clavo".

No es un antipatrón de código enredado como los dos anteriores: es un antipatrón de **criterio de diseño**. Ocurre cuando un equipo (o una persona) domina bien una herramienta y, en vez de evaluar cada problema nuevo por sus propios méritos, fuerza esa misma herramienta una y otra vez.



CONCEPTO · RELACIÓN CON LO VISTO

Un nombre para varias advertencias que ya escucharon

Golden Hammer agrupa, bajo un solo nombre, varias tensiones ya vistas en el curso sin llamarlas antipatrón:

- **DIP (semana 9):** crear una interfaz para cada clase, aunque el detalle nunca vaya a cambiar.
- **Pure Fabrication (semana 10):** fabricar clases artificiales de más, donde una solución simple bastaba.
- **Strategy / Adapter en exceso (semanas 13-14):** encapsular una variación que en la práctica no existe ni se espera.

En el fondo, un patrón aplicado sin que el problema realmente lo amerite es, ni más ni menos, **Golden Hammer** — la solución favorita del equipo, disfrazada de buena práctica.

CONCEPTO

Copy-Paste Programming

Copy-Paste Programming (duplicación de código) es resolver un problema nuevo copiando y pegando código existente con pequeñas modificaciones, en vez de abstraer la lógica común en un solo lugar (una función, una clase, un método de plantilla).

El riesgo principal no es solo el volumen de código repetido: es que, cuando se encuentra un error en la lógica compartida, se corrige en la copia que se está mirando y **no en las demás** — porque nadie recuerda cuántas copias existen ni dónde quedaron.



CONTRAEJEMPLO

El mismo cálculo, copiado tres veces

```
class ReporteMatricula:
    def calcular_total(self, items):
        subtotal = sum(i.precio * i.cantidad for i in items)
        return subtotal * 1.19          # IVA

class ReporteMaterial:
    def calcular_total(self, items):
        subtotal = sum(i.precio * i.cantidad for i in items)
        return subtotal * 1.19          # IVA, copiado y pegado

class ReporteExtension:
    def calcular_total(self, items):
        subtotal = sum(i.precio * i.cantidad for i in items)
        return subtotal * 1.18          # <- copiado con un error: 1.18, no 1.19
```

Cuando cambie la tarifa del IVA, alguien tendrá que recordar que existen tres copias — y, como ya pasó en `ReporteExtension`, una copia puede quedar con un valor distinto sin que nadie lo note. Extraer `calcular_total` a un solo lugar común elimina el riesgo de raíz.

CONCEPTO

Lava Flow

Lava Flow es código muerto o de dudosa utilidad que nadie se atreve a eliminar por miedo a romper algo, porque ya nadie entiende bien para qué servía. El nombre evoca la lava de un volcán: fluyó una vez con un propósito, pero quedó **solidificada** en el sistema mucho después de que ese propósito dejara de existir.

Suele aparecer tras requisitos que cambiaron sin limpiar el código anterior, refactorizaciones que quedaron a medias, o código de prueba/prototipo que migró a producción sin depurarse. Cada bloque de Lava Flow que se acumula hace más lento y más riesgoso cualquier cambio futuro en esa zona del sistema.



TENSIÓN CONCEPTUAL · CIERRE DEL ARCO

Los antipatrones son, casi siempre, principios no aplicados

Ninguno de los cinco antipatrones de hoy es un problema "nuevo" e independiente de lo visto en el curso: cada uno es la sombra de un principio o patrón de las **Unidades 4 y 5** que no se aplicó, o que se aplicó a medias.

- **God Object** ← falta de High Cohesion / SRP.
- **Spaghetti Code** ← falta de un patrón de comportamiento donde hacía falta.
- **Golden Hammer** ← un patrón aplicado sin que el problema lo pidiera.
- **Copy-Paste Programming** ← falta de abstracción de lo común.
- **Lava Flow** ← falta de disciplina de refactorización sostenida.



SÍNTESIS

Los cinco antipatrones de hoy

Antipatrón	Problema central	Lo que lo evita o corrige
God Object	Una clase acumula demasiadas responsabilidades	High Cohesion (GRASP) / SRP (SOLID)
Spaghetti Code	Flujo de control enredado por condicionales anidados	Strategy / State (GoF comportamiento)
Golden Hammer	La misma solución para todos los problemas, sin evaluar si encaja	Aplicar patrones con criterio real, no por costumbre
Copy-Paste Programming	Duplicar código en vez de abstraer lo común	Template Method / extraer una función o clase compartida
Lava Flow	Código muerto que nadie se atreve a eliminar	Refactorización disciplinada y sostenida (Unidad 6)



INTERACCIÓN

¿Cuál antipatrón reconocen en cada caso?

En el chat, para cada caso indiquen cuál de los cinco antipatrones vistos hoy (God Object, Spaghetti Code, Golden Hammer, Copy-Paste Programming o Lava Flow) describe mejor la situación:

1. Una clase `Sistema` tiene 40 métodos, valida usuarios, calcula notas, genera certificados y envía correos — todo en la misma clase.
2. Un módulo de reportes tiene tres funciones casi idénticas que calculan el mismo total, cada una copiada de la anterior con un pequeño ajuste.
3. Nadie del equipo actual sabe para qué sirve un archivo de 300 líneas, pero nadie se atreve a borrarlo por si algo más lo usa.

5 minutos · respondan en el chat

TRANSICIÓN

Ahora, a buscarlos en su propio proyecto

Los cinco antipatrones se vieron con ejemplos de clase. El resto de la sesión es el **laboratorio de cierre de la Unidad 5 completa**: cada equipo revisa su propio diseño del proyecto de Fábrica Escuela buscando antipatrones reales o potenciales, y repasa los patrones GRASP y GoF que ya aplicó a lo largo de las semanas 10 a 14.



INSTRUCCIONES

Laboratorio de cierre — parte 1: antipatronos en su proyecto

Trabajo en equipo — el mismo equipo del proyecto de Fábrica Escuela conformado desde la semana 2.

- Revisen el **diagrama de clases y de secuencia de diseño** actual, junto con el código ya construido de su proyecto.
- Identifiquen **al menos un antipatrón** real o potencial presente en su propio diseño — de los cinco vistos hoy, u otro que reconozcan y sepan justificar.
- Para cada antipatrón identificado, propongan una **corrección concreta**: qué clase, método o responsabilidad cambiaría, y citando explícitamente **qué patrón GRASP o GoF** de los vistos en la unidad (semanas 10 a 14) resolvería el problema.



INSTRUCCIONES

Parte 2: repaso de los patrones ya aplicados en la unidad

- Hagan un **repaso breve** de qué patrones GRASP y/o GoF ya aplicaron a lo largo de la Unidad 5: en el laboratorio de patrones GRASP (semana 11), en el laboratorio de patrones creacionales (semana 12) y en el laboratorio de patrones estructurales (semana 13).
- Para cada patrón aplicado, confirmen si sigue teniendo sentido a la luz de lo visto hoy, o si alguno merece **revisión** — por ejemplo, si al revisarlo hoy reconocen que en realidad fue un caso de Golden Hammer.
- No es necesario rehacer el trabajo ya entregado: es un chequeo de coherencia general del diseño, no una nueva implementación.

Durante el resto de la sesión · trabajo en equipo sobre el proyecto propio

INSTRUCCIONES

Entregable y cierre del laboratorio

- **Formato exacto:** un documento breve (markdown o la herramienta de gestión del proyecto del equipo) que liste el o los **antipatrones identificados** en su propio proyecto, la **corrección propuesta** para cada uno y el **patrón GRASP o GoF** citado para corregirla.
- Debe incluir también el repaso breve de la parte 2: qué patrones ya se aplicaron en la unidad y si alguno quedó marcado para revisión.
- Este análisis es **preparación directa** para el diseño de detalle que el equipo sigue documentando de cara a la recta final del curso — no reemplaza ninguna entrega ya evaluada, es insumo para lo que viene.



SÍNTESIS

Qué debieron lograr en este laboratorio

1. Identificar al menos un antipatrón real o potencial en el diseño actual de su propio proyecto.
2. Proponer una corrección concreta, citando el patrón GRASP o GoF de la unidad que la resuelve.
3. Repasar los patrones GRASP y/o GoF ya aplicados en las semanas 10 a 14, marcando si alguno merece revisión.
4. Dejar todo documentado en un entregable breve, en el repositorio o herramienta de gestión del equipo.

SÍNTESIS · CIERRE DE LA UNIDAD 5

Los veintiséis patrones de la Unidad 5, completos

Familia	Semana	Patrones
GRASP (9)	10	Information Expert, Creator, Controller, Low Coupling, High Cohesion, Polymorphism, Pure Fabrication, Indirection, Protected Variations
Creacionales GoF (5)	12	Factory Method, Abstract Factory, Builder, Prototype, Singleton
Estructurales GoF (6)	13	Adapter, Facade, Composite, Decorator, Proxy, Bridge
Comportamiento GoF (6)	14	Strategy, Observer, Command, State, Template Method, Iterator



CIERRE

Cierre de la Unidad 5

Con esto se cierra la Unidad 5 — Patrones de Diseño y Antipatrones. En cinco semanas se cubrieron los nueve patrones GRASP, los cinco creacionales, los seis estructurales y los seis de comportamiento GoF, y hoy los cinco antipatrones clásicos que aparecen cuando esos principios no se aplican bien — cerrando también el arco abierto desde SOLID en la Unidad 4.

La **semana 15** abre la **Unidad 6 — Calidad en el Diseño de Software**, empezando por el concepto de **deuda técnica** y, luego, código limpio: nombres, funciones, comentarios y refactorización.

